



Documentation

HASOMED MotionSensor 2.0

Version: 11

Date: 11.02.2020

Content

Change history.....	7
Introduction.....	11
Fields of application	11
Product contents	11
Care instructions and service hints	11
Symbols and labels	12
Care instructions	13
Warranty	13
Manufacturer's contact details	14
Operating conditions.....	14
Electromagnetic emission	15
Electromagnetic immunity	15
Protocol basics.....	20
General features.....	20
Bluetooth.....	20
Serial connection	21
Structure of packages.....	21
LED.....	22
USB	22
Commands.....	23
General command structure	31
InitAck.....	31
Sleep	31
ChangeLed	31
SwitchOnOff	31
Error.....	31
Debug	32
Iwrap.....	32
IwrapAck.....	32
SetSensorConfig	32
GetSensorConfigAck.....	32
SetSerial.....	33
GetSerialAck	33
SetPosition.....	33

GetPositionAck	33
SetOriginalPosition	33
GetOriginalPositionAck.....	33
PrepareMeasurementAck	33
StartMeasurementAck	33
StopMeasurementAck.....	34
SetMeasurementMode	34
GetMeasurementModeAck.....	34
SetMeasurementModeExt	34
GetMeasurementModeExtAck.....	35
SetCalibrationData	35
SetCalibrationDataAck.....	36
GetCalibrationData.....	36
GetCalibrationDataAck.....	36
GetScaleFactor	36
GetScaleFactorAck.....	37
GetAvailableMeasurementModesAck	37
SensorStats.....	37
SensorStatsExt.....	37
GetScaleFactorExt	38
ConfigStandBy	38
MeasurementActiveAck	38
GetCurrentSessionIdAck.....	38
GetUsbConnectStateAck	38
GetLinkAck.....	39
ChangeSendStorageMode.....	39
RawData3	39
RawData4	40
RawData5	40
RawData6	41
RawData7	41
RawData8	41
SyncData5.....	41
RawData9	41
RawData10	42

RawData11	42
RawData12	43
RawData13	43
RawData14	44
RawData15	44
RawData16	44
RawData17	45
CalibData	45
OrientationData.....	45
AccComparisonData	45
GaitphaseData	46
MeasPackageRequest.....	46
MeasPackageRequestList	46
MeasPackageRequestRange.....	46
MeasPackageResponse	47
MeasPackageUnavailable	47
SyncPrepare.....	47
SyncTime	47
SyncOffset.....	47
SyncBtClock	48
SyncBtClockAck.....	48
SetRtc.....	48
GetRtcAck	48
SetBtSettings	48
GetBtSettingsAck.....	48
TestConnection.....	49
TestConnectionAck.....	49
ErrorListGetAck.....	49
SdCardGetSectorCountAck.....	49
SdCardGetSessionCountAck	49
SdCardGetSessionInfo	49
SdCardGetSessionInfoAck	50
SdCardClearAll	50
SetMonitorSettings	50
SetOrientationAlgorithmParameter.....	50

GetOrientationAlgorithmParameterAck	50
SetOrientationAlgorithmParameterSingle	51
GetOrientationAlgorithmParameterSingle	51
GetOrientationAlgorithmParameterSingleAck.....	51
SetGaitPhaseAlgorithmParameter	51
GetGaitPhaseAlgorithmParameterAck.....	52
SetGaitPhaseAlgorithmParameterSingle.....	52
GetGaitPhaseAlgorithmParameterSingle	53
GetGaitPhaseAlgorithmParameterSingleAck	53
Measurement mode of sensors	54
Sensor positions	55
Error constants	56
Example of use	57
Measurement packages	59
Availability	59
Bandwidth usage	59
Sample rate fluctuations	59
Conversion of measurement data (packages: RawDataXX)	60
Conversion of LSB values to physical values	60
Apply calibration data	60
Neutral calibration.....	60
Bandwidth estimation RawData5.....	61
Stuffing probability.....	61
Overhead from package structure	61
Used bandwidth	61
Sensor coordinate system	62
Time synchronization	63
Bluetooth clock.....	63
Time duration	63
Operation modes.....	64
External synchronization	64
Low latency optimization	64
Monitoring mode	64
Measurement file format	65
File structure index file.....	65

File structure measurement file	65
Header	65
Calibration data	66
Measurement data	66

Change history

24.01.2012

- Error command: updated list of error codes
- Changed command SetMeasurementMode: updated scale factors

31.01.2012

- Added chapter about coordinate system

27.02.2012

- Changed baud rate from 1500000 to 460800
- Changed command number of Init, InitAck, KeepAlive, KeepAliveAck, ShutDown, ShutDownAck, Reset and ResetAck
- Added command SensorStats

09.03.2012

- Added command Debug
- Added command to read and write calibration data
- Update of error codes

15.05.2012

- Changed command SetCalibrationData/ GetCalibrationData to match changed calibration formulas
- Added command DebugData
- Added command OrientationData

29.05.2012

- Command StopMeasurementAck contains now elapsed time and number of send packages

01.06.2012

- Added command Cmdlwrap

15.06.2012

- Added command ConfigStandBy

21.08.2012

- Conversion for RawData3 updated
- Changed command number of measurement data packages
- Changed command SetMeasurementMode
- Clarified notation of calibration rotation matrix

25.11.2012

- Added new measurement modes (RawData4, RawData5, RawData6, AccComparisonMode)

- Added command PrepareMeasurement and UnPrepareMeasurement
- Added command GetScaleFactor
- Added command GetAvailableMeasurementModes
- Added chapter about typical use of commands
- Update of error codes
- Rotation of sensor axis now done in sensor
- Added description of LED-state

14.12.2012

- Added description for commands SetSerial, GetSerialAck, SetPosition and GetPositionAck
- Added chapter about sensor positioning

21.01.2013

- Changes made for commands PrepareMeasurement, StartMeasurement and StopMeasurement
- Changed command StopMeasurement to stop the measurement timer until a complete data package has been sent
- Added commands for synchronisation using a method similar to Network Time Protocol (NTP) (SyncPrepare, SyncTime, SyncOffset)
- Added commands for synchronisation using Bluetooth clock method (SyncBTClock)
- Chapter about synchronisation added
- Corrected error in description of command RawData5
- Added error constants (ErrorInitMpu9150)
- Added measurement modes RawData7 and RawData8
- Added commands GetBtSettings and SetBtSettings to configure bluetooth settings
- Changed command PrepareMeasurementAck to contain a session id from meas data cache
- Changed commands MeasPackageRequest, MeasPackageResponse and MeasPackageUnavailable to work with session id from meas data cache
- Added command TestConnection
- Changed command SetMeasurementMode to handle data storage and added hint about time requirement
- Removed package number from default header and moved it to all meas data packages as a four byte number
- Added commands CmdSdCardGetSectorCount, CmdSdCardGetSessionCount, CmdSdCardGetSessionInfo and CmdSdCardClearAll
- Added commands CmdGetOriginalPosition and CmdSetOriginalPosition
- Added command CmdChangeLed
- Changed number of command lwrap

07.05.2013

- Corrected byte order from big endian to little endian
- Changed format from SD-card to a Fat-filesystem
- Added chapter about measurement file format of SD-card
- Added additional information for command SdCardClearAll

- Added commands SetRtc and GetRtc
- Added command MeasurementActive
- Added command ChangeSendStorageMode
- Changed stand by behavior
- Changed command Sleep

21.08.2013

- Added commands MeasPackageRequestList and MeasPackageRequestRange
- Added command GetCurrentSessionId
- Added measurement type SyncData5
- Added command SwitchOnOff
- Added chapter about external synchronization
- Added command GetUsbConnectState

07.02.2014

- Added command SetMonitorSettings
- Added command GetLink
- Changed command SdCardGetSectorCount

19.08.2014

- Added command SetSensorConfig/GetSensorConfig
- Added chapter about operating modes

12.01.2016

- Added measurement type RawData9, RawData10, RawData11
- Added support for barometer
- Added commands SetMeasurementModeExt / GetMeasurementModeExt
- Added command SensorStatsExt

23.03.2016

- Added general user informations

21.04.2016

- Added information regarding EN 60601-1-2

01.06.2016

- Added command GetScaleFactorExt
- Added command RawData12

30.01.2017

- Added commands SetOrientationAlgorithmParameter, GetOrientationAlgorithmParameter, SetOrientationAlgorithmParameterSingle, GetOrientationAlgorithmParameterSingle

- Added commands SetGaitPhaseAlgorithmParameter, GetGaitPhaseAlgorithmParameter, SetGaitPhaseAlgorithmParameterSingle, GetGaitPhaseAlgorithmParameterSingle
- Added command GaitPhaseData
- Changed command OrientationData

23.01.2018

- Added RawData13-RawData17

06.06.2018

- Added commands ErrorListGet and ErrorListReset

17.10.2018

- Added information regarding ESD conformity

26.10.2018

- Added information about accessories
- Added more information of Operating conditions
- Corrected Electromagnetic immunity standards

26.11.2018

- Changed chapter Operating conditions

11.02.2020

- Changed name from "MotionSensor" to "MotionSensor 2.0"

Introduction

HASOMED MotionSensor is an IMU (inertial measurement unit) with a 9D chip (accelerometer, gyroscope, magnetic field sensor) that sends data via Bluetooth or USB. Intended use is collection movement data from humans in combination with another device (e.g. RehaGait Analyzer). The sensor has a power button and three multicolor LEDs (center: operating state, left/right: sensor position). All measurement parameters can be adjusted through a communication protocol. An internal memory of 4GB can be used to store measurement data. A Li-Po battery allows at least 8h continuous streaming of measurement data. USB cable and power supply are optional accessories.

Fields of application

The sensor can be integrated into other systems, e.g. RehaGait Analyzer. Use MotionSensor only for intended use as stated for the product combination.

Read the user manual, consisting of instructions for use and operating instructions before using the device. The device may only be used from trained staff.

Product contents

The following list contains external accessories:

- Power supply: XP Power ACM06US05-BB
- USB Cable: Oehlbach USB EVOLUTION C3

Care instructions and service hints

HASOMED MotionSensor is a medical device of class I.



According to 93/42 EEC, HASOMED MotionSensor bears the CE marking.

An inspection of the equipment may only be carried out by HASOMED – service staff. It is not allowed to open the devices. Repairs on the devices may only be carried out by the producer. Do not change battery, it could result in a hazard



The device may not be disposed of in household waste. When returning the device to HASOMED GmbH, the producer takes care of the disposal.

The producer recommends cleaning the HASOMED MotionSensor with a slightly moistened cloth, soaked in water or a disinfectant containing alcohol.

To check and renew the calibration of the sensors it is recommended to make use of HASOMED GmbH's service at least every two years or 2000 measurements.

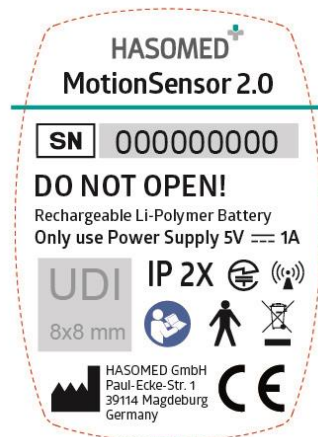
The system may not be used if the sensors:

- Are damaged or open,
- Are noticeable defective, e.g. if the cover is broken,
- Have fallen into water.

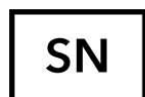
Please immediately contact HASOMED support in such cases.

Symbols and labels

The type label is located on the back of the MotionSensor. This label contains name, serial number, data for the power supply to be used, and the device ID.



The following symbols and labels are used on MotionSensor:



Identifies the device by a unique serial number



Read and obey operating instructions and instructions for use (Referring: ISO 7010-M002)



Do not dispose MotionSensor in normal household waste. For further information on disposal ask HASOMED GmbH.



Requirements for non-ionizing electromagnetic radiation according to IEC 60471-5140 (2003-04) are met



The technical standards of the Ministry of Interior and Communication of Japan (MIC) are met



CE-mark according to 93/42 EEC

IP2X

IP 20 IP-Safety Class (foreign object and protection against water) Referring: DIN EN 60529



Class I medical device without measurement function, the components (Windows Tablet, straps, MotionSensor) of type B being used. RehaGait Analyzer (Pro) is not a system with a medical measurement function according to MEDDEV 2.5 / 1.



Manufacturer Address

Care instructions

Clean the device before each patient contact and before each new application to avoid crosscontamination between patients. Wipe the device surface, as well as all surfaces that you or the patient might be in contact with, with disinfectant.

- The surfaces should be regularly cleaned and disinfected.
- The straps can be washed at 40 ° C. However, they are not suitable for drying.
- Note that the MotionSensor must be removed from the holder before washing. If moisture enters the MotionSensor, it can damage the electronics.
- Never use sharp or corrosive detergents, soaps or dilutions. For heavy soiling that can not be removed, contact the manufacturer.

Warranty

The HASOMED GmbH offers a guarantee on the functioning of the device according to the scope of the description above:

- 2 years after delivery in countries of the European Union.
- 1 year after delivery in countries outside the European Union.

Your warranty claim expires if:

- Damage is due to improper use. This includes, among other situations, the operation of the main unit with hard mechanical objects such as ballpoint pens, damage to the case of the unit or the connector, and falling.
- Devices, others than standard USB components, are electrically connected with the sensor.

Manufacturer's contact details

When making use of the service, please contact the following address or telephone number:

Manufacturing company	HASOMED GmbH Service - RehaGait Paul-Ecke-Str. 1 39114 Magdeburg Germany
Phone	+49-391-61 07 645
Mail	rehagait@hasomed.com
Homepage	www.rehagait.com

Operating conditions

Note about electromagnetic compliance (EMC): HASOMED MotionSensor communicates wireless via Bluetooth 2.1 (2.4GHz band, Effective Radiated Power: +9.84dBm). Many electronic devices may electromagnetic interfere medical devices. This may cause malfunctioning and create unsafe situations. The norm IEC60601-1-2 was introduced to regulate the requirements for EMC. HASOMED MotionSensor complies with norm IEC60601-1-2:2015 regarding immunity and emissions.

HASOMED MotionSensor does not manage essential performances. During normal and intended use no power supply may be connected to prevent electric interference.

Attention:

- Do not use accessories, transducers and cables other than those specified, with the exception of transducers and cables sold by HASOMED, because others may result in increased Emissions or decreased Immunity of HASOMED MotionSensor
- Avoid using devices that may cause strong electromagnetic disturbances near HASOMED MotionSensor

Warning: Use of this equipment adjacent to or stacked with other equipment should be avoided because it could result in improper operation. If such use is necessary, this equipment and the other equipment should be observed to verify that they are operating normally.

Warning: Portable RF communications equipment (including peripherals such as antenna cables and external antennas) should be used no closer than 30 cm (12 inches) to any part of the HASOMED MotionSensor, including cables specified by the manufacturer. Otherwise, degradation of the performance of this equipment could result.

Electromagnetic emission

The "HASOMED MotionSensor" is intended for use in the electromagnetic environment specified below. The customer or the user of the "HASOMED MotionSensor" should assure that it is used in such an environment.

Emissions test	Compliance	Electromagnetic environment - guidance
RF emissions CISPR 11	Group 1	The "HASOMED MotionSensor" uses RF energy only for its internal function. Therefore, its RF emissions are very low and are not likely to cause any interference in nearby electronic equipment.
RF emissions CISPR 11	Class B	The "HASOMED MotionSensor" is suitable for use in all establishments, including domestic establishments and those directly connected to the public low-voltage power supply network that supplies buildings used for domestic purposes.
Harmonic emissions IEC 61000-3-2	Complies	
Voltage fluctuations / flicker emissions IEC 61000-3-3	Complies	

Electromagnetic immunity

The "HASOMED MotionSensor" is intended for use in the electromagnetic environment specified below. The customer or the user of the "HASOMED MotionSensor" should assure that it is used in such an environment.

Attention: An overvoltage pulse during Electrostatic discharge regarding IEC61000-4-2 (15kV air discharge) causes the sensor to be unable to use for a few hours. After that time the sensor recovers and can be used normally again.

Immunity test	IEC 60601 test level	Compliance level	Electromagnetic environment - guidance
Electrostatic discharge (ESD) IEC 61000-4-2	Contact Discharge: ± 8 kV Air Discharge: ± 2 kV, ± 4 kV, ± 8 kV, ± 15 kV	± 8 kV contact ± 15 kV air	Floors should be wood, concrete or ceramic tile. If floors are covered with synthetic material, the relative humidity should be at least 30%.
Radiated RF EM field IEC 61000-4-3	80-2700 MHz; 1kHz AM 80 %; 3 V/m	3 V/m	

Proximity fields from RF wireless communications equipment IEC 61000-4-3	385 MHz; Pulse Modulation: 18 Hz; 27 V/m 450 MHz, Pulse Modulation: 18 Hz; 1 kHz sine; 28 V/m 710, 745, 780 MHz; Pulse Modulation: 217 Hz; 9 V/m 810, 870, 930 MHz; Pulse Modulation: 18 Hz; 28 V/m 1720, 1845, 1970 MHz; Pulse Modulation: 217 Hz; 28 V/m 2450 MHz; Pulse Modulation: 217 Hz; 28 V/m; 5240, 5500, 5785 MHz; Pulse Modulation: 217 Hz; 9 V/m	27 V/m 28 V/m 9 V/m 28 V/m 28 V/m 28 V/m 9 V/m	
Electrical fast transient/burst IEC 61000-4-4	Power lines: 2 kV; 100 kHz repetition frequency Signal lines: 1 kV; 100 kHz repetition frequency	AC port L+N ± 2 kV USB cable ± 1 kV	Mains power quality should be that of a typical commercial or hospital environment.
Surge IEC 61000-4-5	L-PE and N-PE: 2kV L-N: 1kV	L-N ± 1 kV	Mains power quality should be that of a typical commercial or hospital environment.
Conducted disturbances induced by RF fields IEC 61000-4-6	0.15-80 MHz; 1kHz AM 80 %; 3 Vrms , 6 Vrms in ISM band	150kHz - 80MHz 3 V	
Rated power frequency magnetic fields IEC 61000-4-8	30 A/m, 50 Hz and 60 Hz	30 A/m	Power frequency magnetic fields should be at levels characteristic of a typical location in a typical commercial or hospital environment.
Voltage dips, short interruptions and voltage variations on	0 % UT for 0.5 cycle at 8 phase angles 0 % UT for 1 cycle at 0° 70 % UT for 25/30 cycles at 0°	<5 % UT (>95 % dip in UT) for 0.5 cycle	Mains power quality should be that of a typical commercial or hospital environment. If the user of the "HASOMED MotionSensor" requires

power supply input lines IEC 61000-4-11	0 % UT for 250/300 cycles 0°	40 % UT (60 % dip in UT) for 5 cycles 70 % UT (30 % dip in UT) for 25 cycles <5 % UT (>95 % dip in UT) for 5 sec	continued operation during power mains interruptions, it is recommended that the "HASOMED MotionSensor" is powered from an uninterruptible power supply or battery.
NOTE UT is the a.c. mains voltage prior to application of the test level.			

Guidance and manufacturer's declaration - electromagnetic immunity			
The "HASOMED MotionSensor" is intended for use in the electromagnetic environment specified below. The customer or the user of the "HASOMED MotionSensor" should assure that it is used in such an environment.			
Immunity test	IEC 60601 test level	Compliance level	Electromagnetic environment - guidance
Conducted RF IEC 61000-4-6	3 Vrms 150 kHz to 80 MHz	3 V	Portable and mobile RF communications equipment should not be used no closer than 30 cm (12 inches) to any part of the "HASOMED MotionSensor", including cables, than the recommended separation distance calculated from the equation applicable to the frequency of the transmitter. Recommended separation distance d = 1.2vP
Radiated RF IEC 61000-4-3	3 V/m 80 MHz to 2.5 GHz	3 V/m	d = 1.2vP 80 MHz to 800 MHz d = 2.3vP 800 MHz to 2.5 GHz where P is the maximum output power rating of the transmitter in watts (W) according to the transmitter

			manufacturer and d is the recommended separation distance in meters (m). Field strengths from fixed RF transmitters, as determined by an electromagnetic site survey, ^a should be less than the compliance level in each frequency range.^b Interference may occur in the vicinity of equipment marked with the following symbol: 
NOTE 1 At 80 MHz and 800 MHz, the higher frequency range applies. NOTE 2 These guidelines may not apply in all situations. Electromagnetic propagation is affected by absorption reflection from structures, objects and people.			
^a Field strengths from fixed transmitters such as base stations for radio (cellular/cordless) telephones, land mobile radios, amateur radio, AM and FM radio broadcast and TV broadcast, cannot be predicted theoretically with accuracy. To assess the electromagnetic environment due to fixed RF transmitters an electromagnetic site survey should be considered. If the measured field strength in the location in which the "HASOMED MotionSensor" is used, exceeds the applicable RF compliance level above, the "HASOMED MotionSensor" should be observed to verify normal operation. If abnormal performance is observed, additional measures may be necessary, such as reorienting or relocating the "HASOMED MotionSensor". ^b Over the frequency range 150 kHz to 80 MHz, field strength should be less than 3 V/m.			

Recommended separation distances between portable and mobile RF communications equipment and the "HASOMED MotionSensor"			
The "HASOMED MotionSensor" is intended for use in the electromagnetic environment in which radiated RF disturbances are controlled. The customer or the user of the "HASOMED MotionSensor" can help prevent electromagnetic interference by maintaining a minimum distance between portable and mobile RF communications equipment (transmitters) and the "HASOMED MotionSensor" as recommended below, according to the maximum output power of the communication equipment.			
Rated maximum output power of	Separation distance according to frequency of transmitter		
	m		
	150 kHz to 80 MHz	80 MHz to 800 MHz	800 MHz to 2.5 GHz

transmitter W	$d = 1.2\sqrt{P}$	$d = 1.2\sqrt{P}$	$d = 2.3\sqrt{P}$
0.01	0.12	0.12	0.23
0.1	0.38	0.38	0.73
1	1.2	1.2	2.3
10	3.8	3.8	7.3
100	12	12	23

For transmitters rated at a maximum output power not listed above, the recommended separation distance d in metres (m) can be determined using the equation applicable to the frequency of the transmitter, where P is the maximum output power rating of the transmitter in watts (W) according to the transmitter manufacturer.

NOTE 1 At 80 MHz and 800 MHz, the separation distance for the higher frequency range applies.

NOTE 4 These guidelines may not apply in all situations. Electromagnetic propagation is affected by absorption and reflection from structures, objects and people.

Protocol basics

The transmission protocol describes the bidirectional communication between a PC and a sensor using a serial connection. The communication is package-based, i.e. all transmitted data is packed in defined packages. At each time a sensor is in a defined state. The state change is done via commands. Depending on the current state not all packages are allowed. The communication is initiated by a PC. The sensor only responds reactively to commands. Each sensor can be identified by a unique serial number (see label on sensor case). The sensor provides several predefined operating modes

General features

- Size: 47 x 38 x 17 mm (W x H x D):
- Weight: 34 g
- (Optional) mounting straps

- Internal memory: 4 GB
- 3D gyroscope, 3D accelerometer, 3D magnetometer
- Up to 600 Hz sampling rate
- USB mass storage device

- Operating time (continuous streaming): ~7 h
- Operating time (without streaming): ~10 h
- Charging time: ~2h for 80% capacity, ~6h for 100% capacity

- Bluetooth 2.1 EDR
- Transmission range up to 30m
- Up to 7 in a wireless network

Bluetooth

The connection with a sensor is made via Bluetooth. As profile the Serial Port Profile (SPP) is used. To set up the connection, it is required to pair the sensor with the other device utilizing Secure Simple Pairing (SSP). After pairing a (virtual) COM port or socket is created, this is used for data transmission. It is advised to not use serial port, use direct Bluetooth programming instead.

Serial connection

These are the parameters for serial connection:

Baud rate	460800 Bit/s
Data bits	8
Stop bits	1
Parity	0
Flow control	None

Hint: Com ports created by Bluetooth-stacks are “virtual” com ports and often ignore the parameters above. Sometimes they accept only standard baud rate of 115200 Bit/s.

Structure of packages

Each package consists of a fixed header and variable data depending on the command. In order to differentiate distinctly packages from data stream, each packet has a start and stop byte. After the start byte follows a 2 byte checksum (for error detection, sum of all command number and data bytes, calculated for unstuffed package). A package has a maximum data length of 384 bytes. To avoid misinterpretations special bytes (see constants) are stuffed, that means they are preceded by a special stuff byte (like escaping chars when programming) and the byte itself is modified with a stuffing key using exclusive or (XOR). The byte order is little-endian.

The structure of all packages is as follows:

Start byte 1 byte	Checksum 2 bytes	Command 2 bytes	Data ≤ 384 bytes	Stop byte 1 byte
<- Stuffing ->				
<- Checksum ->				

Checksum

- 2 bytes length
- Sum of not stuffed command and data

Command

- 2 bytes length
- Each command has an unique number
- Response for a command is command number plus one

Data

- At maximum 384 bytes long

The special bytes are the following:

Constant	Hex	Binary
Start byte	0xF0	1111 0000
Stop byte	0x0F	0000 1111
Stuffing byte	0x81	1000 0001
Stuffing key	0x55	01010101

Stuffing example:

Description	Pre stuff package in hex	Post stuff package in hex
SetSerial command with serial number decimal 240 (hex: 0xF0)	F0 30 01 40 00 F0 00 00 00 0F	F0 30 01 40 00 81 A5 00 00 00 0F

LED

The LED indicates the current state of the sensor. After turning on the center sensor LED light is on as long as the initialization takes. After initialization the sensor is in idle mode. Following table describes all blink modes:

Mode	Blink behavior	Color of status LED
Init	Led on	Green
Idle	Slow blinking	Green
Measurement with connection	Led on with one short off period	Green
Charging	Slow blinking	Blue
Fully charged	Led on	Blue
Low battery	Blinking	Orange
Error	Fast blinking	Red

USB

The sensor can be connected via USB-C cable to a PC and is recognized as a CDC device. If USB cable is plugged in, an active measurement is stopped. It is possible to start and (normally) stop a measurement when the sensor is connected, but only without storing measurement data because the sensor unmounts its own file system during USB connection.

To use the sensor as CDC device a driver (.inf) file is required, it can be obtained from HASOMED. The CDC functionality provides a serial port, which can be used for interaction with the sensor, equally to the Bluetooth connection.

Commands

The following table summarizes all commands and shows command number, name, short description and sender/receiver. Sender and receiver are abbreviated by S and R. Most of the commands are send from Master (=M, Data logger/smart device/PC) to the sensor (=S), if both sides send a command, then B is used.

The communication is done for most commands (except the measurement data packets) with an acknowledgment. This means that for command XXX a response XXXAck is sent. If the command raises an error, instead of an acknowledgment an error package is sent with an error code. The command number of XXXAck packet always follows the rule command number XXX plus one.

Connection

Command number decimal	Command number hex	Command name	Short description	S	R
0	0x00	Init	Init connection, all settings are set to default	M	S
1	0x01	InitAck	Response for Init	S	M
2	0x02	KeepAlive	Keep connection alive, resets StandBy-Timer	M	S
3	0x03	KeepAliveAck	Response for KeepAlive	S	M
4	0x04	ShutDownSensor	Turn sensor off	M	S
5	0x05	ShutDownSensorAck	Response for ShutDownSensor	S	M
6	0x06	Reset	Resets sensor	M	S
7	0x07	ResetAck	Response for Reset	S	M
8	0x08	Sleep	Puts Bluetooth module into sleep mode	M	S
9	0x09	SleepAck	Response for Sleep	S	M
20	0x14	ChangeLed	Changes Led state to fast blinking for a specified time	M	S
21	0x15	ChangeLedAck	Response for ChangeLed	S	M
22	0x16	SwitchOnOff	Change on-off behaviour	M	S
23	0x17	SwitchOnOffAck	Response for SwitchOnOff	S	M

Error state

Command number decimal	Command number hex	Command name	Short description	S	R
44	0x2C	Error	Error	S	M
48	0x30	Debug	Debug-message	S	M
50	0x32	Iwrap	iWRAP command	B	S

Configuration

Command number decimal	Command number hex	Command name	Short description	S	R
60	0x3C	SetSensorConfig	Set sensor config	M	S
61	0x3D	SetSensorConfigAck	Response for SetSensorConfig	S	M
62	0x3E	GetSensorConfig	Get sensor config	M	S
63	0x3F	GetSensorConfigAck	Response for GetSensorConfig	S	M
64	0x40	SetSerial	Set serial number	M	S
65	0x41	SetSerialAck	Response for SetSerial	S	M
66	0x42	GetSerial	Get serial number	M	S
67	0x43	GetSerialAck	Response for GetSerial	S	M
68	0x44	SetSensorPosition	Set sensor position	M	S
69	0x45	SetSensorPositionAck	Response for SetSensorPosition	S	M
70	0x46	GetSensorPosition	Get sensor position	M	S
71	0x47	GetSensorPositionAck	Response for GetSensorPosition	S	M
72	0x48	SetOriginalSensorPosition	Set original sensor position (factory default)	M	S
73	0x49	SetOriginalSensorPositionAck	Response for SetOriginalSensorPosition	S	M
74	0x4A	GetOriginalSensorPosition	Set original sensor position (factory default)	M	S
75	0x4B	GetOriginalSensorPositionAck	Response for GetOriginalSensorPosition	S	M
76	0x4C	SetCalibrationData	Write calibration data	M	S
77	0x4D	SetCalibrationDataAck	Response for SetCalibrationData	S	M
78	0x4E	GetCalibrationData	Read calibration data	M	S
79	0x4F	GetCalibrationDataAck	Response for GetCalibrationData	S	M
84	0x54	GetScaleFactor	Get scale factor for RawDataXX measurement modes	M	S
85	0x55	GetScaleFactorAck	Response for GetScaleFactor	S	M
86	0x56	GetAvailableMeasurementModes	Get available measurement modes	M	S
87	0x57	GetAvailableMeasurementModesAck	Response for GetAvailableMeasurementModes	S	M
88	0x58	PrepareMeasurement	Prepare measurement, must be called before <i>StartMeasurement</i> , after this command measurement mode cannot be changed until <i>UnPrepareMeasurement</i> or <i>StopMeasurement</i>	M	S
89	0x59	PrepareMeasurementAck	Response for PrepareMeasurement	S	M
90	0x5A	UnPrepareMeasurement	Unprepare measurement	M	S
91	0x5B	UnPrepareMeasurementAck	Response for UnPrepareMeasurement	S	M
92	0x5C	StartMeasurement	Starts a measurement	M	S

93	0x5D	StartMeasurementAck	Response for StartMeasurement	S	M
94	0x5E	StopMeasurement	Stops a measurement	M	S
95	0x5F	StopMeasurementAck	Response for StopMeasurement	S	M
96	0x60	SetMeasurementMode	Set measurement mode	M	S
97	0x61	SetMeasurementModeAck	Response for SetMeasurementMode	S	M
98	0x62	GetMeasurementMode	Get measurement mode	M	S
99	0x63	GetMeasurementModeAck	Response for GetMeasurementMode	S	M
100	0x64	SensorStats	Provides battery, RSSI and BER	M	S
101	0x65	SensorStatsAck	Response for SensorStats	S	M
102	0x66	ConfigStandBy	Configures StandBy time	M	S
103	0x67	ConfigStandByAck	Response for ConfigStandBy	S	M
104	0x68	MeasurementActive	Check if measurement is active	M	S
105	0x69	MeasurementActiveAck	Response for MeasurementActive	S	M
106	0x6A	ChangeSendStorageMode	Change SendStorageMode during active measurement	M	S
107	0x6B	ChangeSendStorageModeAck	Response for ChangeSendStorageMode	S	M
108	0x6C	GetCurrentSessionId	Retrieve current session id	M	S
109	0x6D	GetCurrentSessionIdAck	Response for GetCurrentSessionId	S	M
110	0x6E	GetUsbConnectState	Retrieve usb connect state	M	S
111	0x6F	GetUsbConnectStateAck	Response for GetUsbConnectState	S	M
112	0x70	GetLink	Retrieve link (Bluetooth or USB-CDC)	M	S
113	0x71	GetLinkAck	Response for GetLink	S	M
114	0x72	SetMeasurementModeExt	Set measurement mode	M	S
115	0x73	SetMeasurementModeExtAck	Response for SetMeasurementModeExt	S	M
116	0x74	GetMeasurementModeExt	Get measurement mode	M	S
117	0x75	GetMeasurementModeExtAck	Response for GetMeasurementModeExt	S	M
118	0x76	SensorStatsExt	Get sensor stats	M	S
119	0x77	SensorStatsExtAck	Response for SensorStatsExt	S	M
120	0x78	GetScaleFactorExt	Get scale factor for RawDataXX measurement modes (extended version)	M	S
121	0x79	GetScaleFactorExtAck	Response for GetScaleFactorExt	S	M

Measurement data

Command number decimal	Command number hex	Command name	Short description	S	R
------------------------	--------------------	--------------	-------------------	---	---

125	0x7D	RawData3	Raw values of sensor chips (AccX, AccY, AccZ, GyroX, GyroY, GyroZ, MagX, MagY, MagZ) minimal packed in 16 samples per package	S	M
127	0x7F	DebugData	Raw values of sensor chips with temperature (1 sample per package)	S	M
128	0x80	CalibData	Calibrated sensor values	S	M
129	0x81	OrientationData	Orientation of sensor as quaternion	S	M
130	0x82	RawData4	Raw values of sensor chips (AccX, AccY, AccZ, GyroX, GyroY, GyroZ, MagX, MagY, MagZ) minimal packed in 6 samples per package	S	M
131	0x83	RawData5	Raw values of sensor chips (AccX, AccY, AccZ, GyroX, GyroY, GyroZ, MagX, MagY, MagZ) minimal packed in 14 samples per package	S	M
132	0x84	RawData6	Raw values of sensor chips (AccX, AccY, AccZ, GyroX, GyroY, GyroZ, MagX, MagY, MagZ) minimal packed in 6 samples per package	S	M
133	0x85	AccComparisonData	Raw values of Acc sensor chips for comparison (MPU6050 {AccX, AccY, AccZ}, BMA180 {AccX, AccY, AccZ}) minimal packed in 16 samples per package	S	M
134	0x86	RawData7	Raw values of sensor chips (AccX, AccY, AccZ, GyroX, GyroY, GyroZ, MagX, MagY, MagZ) minimal packed in 1 samples per package	S	M
135	0x87	RawData8	Raw values of sensor chips (AccX, AccY, AccZ, GyroX, GyroY, GyroZ, MagX, MagY, MagZ) plus a timestamp minimal packed in 1 samples per package	S	M
136	0x88	SyncData5	Structure equal to RawData5, but clock used from external source	S	M
137	0x89	RtData1	Do not use	S	M
138	0x8A	RawData9	Raw values of sensor chips (AccX, AccY, AccZ, GyroX, GyroY, GyroZ, MagX, MagY, MagZ) minimal packed in 14 samples per package	S	M

139	0x8B	RawData10	Raw values of sensor chips (AccX, AccY, AccZ, GyroX, GyroY, GyroZ, MagX, MagY, MagZ) minimal packed in 1 samples per package	S	M
140	0x8C	RawData11	Raw values of sensor chips (AccX, AccY, AccZ, GyroX, GyroY, GyroZ, MagX, MagY, MagZ, Pressure) minimal packed in 1 samples per package	S	M
141	0x8D	RawData12	Raw values of sensor chips (AccX, AccY, AccZ, GyroX, GyroY, GyroZ, MagX, MagY, MagZ, Temp) minimal packed in 12 samples per package	S	M
142	0x8E	GaitPhaseData	Gait phase	S	M
143	0x8F	RawData13	Raw values of sensor chips (AccX, AccY, AccZ, GyroX, GyroY, GyroZ, MagX, MagY, MagZ, Temp) minimal packed in 1 samples per package	S	M
144	0x90	RawData14	Raw values of sensor chips (AccX, AccY, AccZ, GyroX, GyroY, GyroZ,) minimal packed in 21 samples per package	S	M
145	0x91	RawData15	Raw values of sensor chips (AccX, AccY, AccZ, GyroX, GyroY, GyroZ,) minimal packed in 1 samples per package	S	M
146	0x92	RawData16	Raw values of sensor chips (AccX, AccY, AccZ, GyroX, GyroY, GyroZ, Temp) minimal packed in 18 samples per package	S	M
147	0x93	RawData17	Raw values of sensor chips (AccX, AccY, AccZ, GyroX, GyroY, GyroZ, Temp) minimal packed in 1 samples per package	S	M

Data storage

Command number decimal	Command number hex	Command name	Short description	S	R
------------------------	--------------------	--------------	-------------------	---	---

150	0x96	MeasPackageRequest	Request retransmission of a measurement data package	M	S
151	0x97	MeasPackageResponse	Positive Response for MeasPackageRequest	S	M
152	0x98	MeasPackageUnavailable	Negative Response for MeasPackageRequest	S	M
154	0x9A	MeasPackageRequestList	Request retransmission of multiple measurement data package (list of package numbers)	M	S
156	0x9C	MeasPackageRequestRange	Request retransmission of multiple measurement data package (range of package numbers)	M	S

Time synchronization

Command number decimal	Command number hex	Command name	Short description	S	R
160	0xA0	SyncPrepare	Test time	M	S
161	0xA1	SyncPrepareAck	Response for SyncPrepare	S	M
162	0xA2	SyncTime	Set time	M	S
163	0xA3	SyncTimeAck	Response for SyncTime	S	M
164	0xA4	SyncOffset	Set time offset	M	S
165	0xA5	SyncOffsetAck	Response for SyncOffset	S	M
166	0xA6	SyncBtClock	Synchronize time by using Bluetooth clock	M	S
167	0xA7	SyncBtClockAck	Response for SyncBtClock	S	M
168	0xA8	SetRtc	Set real time clock	M	S
169	0xA9	SetRtcAck	Response for SetRtc	S	M
170	0xAA	GetRtc	Get real time clock	M	S
171	0xAB	GetRtcAck	Response for GetRtc	S	M

Bluetooth settings

Command number decimal	Command number hex	Command name	Short description	S	R
176	0xB0	SetBtSettings	Set maximum transfer unit (MUT) and role (master/slave)	M	S
177	0xB1	SetBtSettingsAck	Response for	S	M
178	0xB2	GetBtSettings	Get maximum transfer unit (MUT) and role (master/slave)	M	S
179	0xB3	GetBtSettingsAck	Response for	S	M
180	0xB4	TestConnection	Test connection by requesting a response with variable data length	M	S

181	0xB5	TestConnectionAck	Response for TestConnection	S	M
182	0xB6	SetCalibrationDataExt	Set extended calibration data	M	S
183	0xB7	SetCalibrationDataExtAck	Response for SetCalibrationDataExt	S	M
184	0xB8	GetCalibrationDataExt	Get extended calibration data	M	S
185	0xB9	GetCalibrationDataExtAck	Response for GetCalibrationDataExt	S	M
186	0xBA	ErrorListGet	Get list of internal errors	M	S
187	0xBB	ErrorListGetAck	Response for ErrorListGet	S	M
188	0xBC	ErrorListReset	Reset list of internal errors	M	S
189	0xBD	ErrorListResetAck	Response for ErrorListReset	S	M

Internal storage

Command number decimal	Command number hex	Command name	Short description	S	R
192	0xC0	SdCardGetSectorCount	Get number of sector from SD-card (1 sector = 512 Byte), if sector count is zero, then there is no SD-card inserted	M	S
193	0xC1	SdCardGetSectorCountAck	Response for SdCardGetSectorCount	S	M
194	0xC2	SdCardGetSessionCount	Get number of measurement sessions	M	S
195	0xC3	SdCardGetSessionCountAck	Response for SdCardGetSessionCount	S	M
196	0xC4	SdCardGetSessionInfo	Get information for a measurement session	M	S
197	0xC5	SdCardGetSessionInfoAck	Response for SdCardGetSessionInfo	S	M
198	0xC6	SdCardClearAll	Deletes all measurement sessions stored on SD-card	M	S
199	0xC7	SdCardClearAllAck	Response for SdCardClearAll	S	M

Monitor settings

Command number decimal	Command number hex	Command name	Short description	S	R
208	0xD0	SetMonitorSettings	Sets RTC and puts the sensor in sleep forever until connect to USB	M	S

Algorithms

Command number decimal	Command number hex	Command name	Short description	S	R
224	0xE0	SetOrientationAlgorithmParameter	Set all parameters for orientation algorithm	M	S
225	0xE1	SetOrientationAlgorithmParameterAck	Response for SetOrientationAlgorithmParameter	M	S
226	0xE2	GetOrientationAlgorithmParameter	Get all parameters for orientation algorithm	M	S
22	0xE3	GetOrientationAlgorithmParameterAck	Response for GetOrientationAlgorithmParameter	M	S
228	0xE4	SetOrientationAlgorithmParameterSingle	Set single parameter for orientation algorithm	M	S
229	0xE5	SetOrientationAlgorithmParameterSingleAck	Response for SetOrientationAlgorithmParameterSingle	M	S
230	0xE6	GetOrientationAlgorithmParameterSingle	Get single parameter for orientation algorithm	M	S
231	0xE7	GetOrientationAlgorithmParameterSingleAck	Response for GetOrientationAlgorithmParameterSingle	M	S
232	0xE8	SetGaitPhaseAlgorithmParameter	Set all parameters for gait phase algorithm	M	S
233	0xE9	SetGaitPhaseAlgorithmParameterAck	Response for SetGaitPhaseAlgorithmParameter	M	S
234	0xEA	GetGaitPhaseAlgorithmParameter	Get all parameters for gait phase algorithm	M	S
235	0xEB	GetGaitPhaseAlgorithmParameterAck	Response for GetGaitPhaseAlgorithmParameter	M	S
236	0xEC	SetGaitPhaseAlgorithmParameterSingle	Set single parameter for gait phase algorithm	M	S
237	0xED	SetGaitPhaseAlgorithmParameterSingleAck	Response for SetGaitPhaseAlgorithmParameterSingleAck	M	S
238	0xEE	GetGaitPhaseAlgorithmParameterSingle	Get single parameter for gait phase algorithm	M	S
239	0xEF	GetGaitPhaseAlgorithmParameterSingleAck	Response for GetGaitPhaseAlgorithmParameterSingle	M	S

General command structure

General structure of each command in bytes:

0	Start byte	
1	Checksum	
2	Checksum	
3	Command number	
4	Command number	
5-X	Data	Depending on command
X+1	Stop byte	

The detailed description of the data packages below starts at 0 with counting, in complete package it is accordingly byte 5. All commands that are missing from the following list have no data.

InitAck

0..3	Version	Version of protocol
4..7	Text length Welcome text	
8..X	Welcome text	

Sleep

After receiving this command, the sensor sends a response and waits for 1 second. After that time, the sensor closes the connection (if parameter “close connection” is set to 1) and puts the Bluetooth module to sleep for duration in ms. During sleep time the Bluetooth module rejects every connection request.

0..3	Duration	In ms
4..7	Close connection	1 = close connection, 0 = do not close connection

ChangeLed

0..3	Duration	Time in ms how long LED will blink very fast
------	----------	--

SwitchOnOff

This command allows enabling or disabling the possibility to turn the sensor off when power switch is pressed. By default the sensor can be turned off.

0..3	On-off	0: sensor can be turned off (default), 1: sensor cannot be turned off
------	--------	---

Error

This package indicates an error.

0..3	Error code	See chapter Error constants
------	------------	-----------------------------

Debug

Attention: only for test cases

This package contains a text message for debugging purposes and is disabled in all release versions of the firmware.

0..1	Text length	Length with without 0 termination
2..X	Text	Text, no 0 termination at the end

lwrap

Attention: only for test cases

This command allows execution of iWRAP-commands for the Bluetooth module. The maximum length of command text is limited to 512 chars.

0..1	Text length	Length with without 0 termination
2..X	Text	Text, no 0 termination at the end

lwrapAck

Attention: only for test cases

This command is the response for *lwrap*. The maximum size of command text is limited to 512 chars.

0..1	Text length	Length with without 0 termination
2..X	Text	Text, no 0 termination at the end

SetSensorConfig

This command changes the sensor configuration. At the moment only the operating mode can be changed from outside. Depending on the chosen operating mode, this command changes several iWRAP settings and needs around 2 seconds to send the response. Because of legal reason, the user can not change the sensor configuration, so this command is protected with a password.

0..3	Operating mode	Mode as 32bit mask See chapter Operation modes
4..7	Password	
8..11	Password	
12..15	Password	

GetSensorConfigAck

This command retrieves the sensor configuration and operating mode.

0..3	Sensor configuration	Sensor configuration as reported in init text message
4..7	Operating mode	Configuration as 32bit mask (See chapter Operation modes)

SetSerial

This command changes the serial number, changes the iWRAP Bluetooth device name and needs around 1 second to send the response. Because of legal reason, the user can not change the serial number, so this command is protected with a password.

0..3	Serial number	As integer
4..7	Password	
8..11	Password	
12..15	Password	

GetSerialAck

0..3	Serial number	
------	---------------	--

SetPosition

0..3	Position des Sensor	See chapter Sensor positions
------	---------------------	------------------------------

GetPositionAck

0..3	Position des Sensor	See chapter Sensor positions
------	---------------------	------------------------------

SetOriginalPosition

This command changes the original position (this position should be identical to the slide on the sensor). Because of legal reason, the user can not change the serial number, so this command is protected with a password.

0..3	Original sensor position	See chapter Sensor positions
4..7	Password	
8..11	Password	
12..15	Password	

GetOriginalPositionAck

0..3	Original sensor position	See chapter Sensor positions
------	--------------------------	------------------------------

PrepareMeasurementAck

This command is the positive response of command PrepareMeasurement when everything (Bluetooth clock sync, package counter reset, SD-card session started) was successfully prepared. The session id is necessary to request lost packages after measurement was stopped.

0..3	Session Id	Id of current measurement session
------	------------	-----------------------------------

StartMeasurementAck

This command is the positive response for StartMeasurement und contains the start time in ms from sensor.

0..3	Time	Start time in ms from sensor
------	------	------------------------------

StopMeasurementAck

This command is the positive response for StopMeasurement and contains the stop time in ms and package count from sensor.

0..3	Time	Stop time in ms from sensor
4..7	Number of packages	Package count (Hint: in dependency of used measurement mode one package can contain multiple samples)

SetMeasurementMode

This command sets the measurement mode. The sensor must be unprepared. The available modes can be queried via GetAvailableMeasurementModes command. The values for MeasurementMode , AccMode, GyroMode, MagMode and SendStorageMode can be found under Measurement mode of sensors. During execution of SetMeasurementMode all internal sensors may be reinitialized, so the response can take some time (around 250ms).

0..3	Sample rate	In Hz, maximum: 600Hz
4..7	MeasurementMode	Command number of measurement data mode (RawData5, ...)
8..11	AccMode	Mode of accelerometer
12..15	GyroMode	Mode of gyroscope
16..19	MagMode	Mode of magnetic field sensor
20..23	SendStorageMode	Send and storage mode

GetMeasurementModeAck

This command gets the current measurement mode. The values for MeasurementMode , AccMode, GyroMode, MagMode and SendStorageMode can be found under Measurement mode of sensors.

0..3	Sample rate	In Hz, maximum: 600Hz
4..7	MeasurementMode	Command number of measurement data mode (RawData5, ...)
8..11	AccMode	Mode of accelerometer
12..15	GyroMode	Mode of gyroscope
16..19	MagMode	Mode of magnetic field sensor
20..23	SendStorageMode	Send and storage mode

SetMeasurementModeExt

This command sets the measurement mode. It is a replacement for SetMeasurementMode and adds pressure mode. The sensor must be unprepared. The available modes can be queried via GetAvailableMeasurementModes command. The values for MeasurementMode , AccMode, GyroMode, MagMode, PressureMode and SendStorageMode can be found under Measurement mode of sensors. During execution of SetMeasurementMode all internal sensors may be reinitialized, so the response can take some time (around 250ms).

0..3	Sample rate	In Hz, maximum: 600Hz
4..7	MeasurementMode	Command number of measurement data mode (RawData5, ...)
8..11	AccMode	Mode of accelerometer
12..15	GyroMode	Mode of gyroscope
16..19	MagMode	Mode of magnetic field sensor
20..23	PressureMode	Mode of pressure sensor
24..27	SendStorageMode	Send and storage mode

GetMeasurementModeExtAck

This command gets the current (extended) measurement mode. The values for MeasurementMode , AccMode, GyroMode, MagMode, PressureMode and SendStorageMode can be found under Measurement mode of sensors.

0..3	Sample rate	In Hz, maximum: 600Hz
4..7	MeasurementMode	Command number of measurement data mode (RawData5, ...)
8..11	AccMode	Mode of accelerometer
12..15	GyroMode	Mode of gyroscope
16..19	MagMode	Mode of magnetic field sensor
20..23	PressureMode	Mode of pressure sensor
24..27	SendStorageMode	Send and storage mode

SetCalibrationData

This command writes calibration data to the flash memory of the sensor. The values for bias, rotation matrix and Cg1/Cg2 are 4-byte long signed integers scaled by factor 1.000.000 (1000*1000). If writing was not successful, an error package with error code *ErrorCalibrationWrite* is sent instead of *SetCalibrationDataAck*. Possible values for channel and mode can be found in chapter Measurement mode of sensors. Because of limited size of flash memory only the following channel / modes combinations are valid.

Channel	Mode
0	3 (8g)
0	4 (16g)
1	2 (1000°/s)
1	3 (2000°/s)
2	1 (1,3ga)

The conversion from raw values to calibrated values can be found in chapter Conversion of measurement data (packages: RawDataXX).

0..3	Channel	
4..7	Mode	
8..11	Bias X	Bias values scaled by factor
12..15	Bias Y	
16..19	Bias Z	

20..23	Matrix XX	Matrix values scaled by factor
24..27	Matrix XY	
28..31	Matrix XZ	
32..35	Matrix YX	
36..39	Matrix YY	
40..43	Matrix YZ	
44..47	Matrix ZX	
48..51	Matrix ZY	
52..55	Matrix ZZ	
56..59	Cg1 X	Cg1 values scaled by factor
60..63	Cg1 Y	
64..67	Cg1 Z	
68..71	Cg2 X	Cg2 values scaled by factor
72..75	Cg2 Y	
76..79	Cg2 Z	

SetCalibrationDataAck

This command is the response for *SetCalibrationData* if writing calibration data was successful.

0..3	Channel	
4..7	Mode	

GetCalibrationData

This command requests calibration data from the flash memory of the sensor. The values of bias, rotation matrix and Cg1/Cg2 are scaled by factor 1.000.000 (1000*1000) to avoid floating values. If no calibration data for the channel / mode is available, an error package with error number *ErrorCalibrationRead* is sent instead of *GetCalibrationDataAck*. Additional information about channel and mode can be found in chapter Measurement mode of sensors.

0..3	Channel	
4..7	Mode	

GetCalibrationDataAck

This command is the response for *GetCalibrationData*. The structure is identical to *SetCalibrationData*

GetScaleFactor

This command retrieves scale factors for each sensor chip. Possible values for channel and mode can be found in chapter Measurement mode of sensors.

0..3	Channel	
4..7	Mode	

GetScaleFactorAck

This command is the response for *GetScaleFactor*. Channel and mode are the same values as in *GetScaleFactor*.

0..3	Channel	
4..7	Mode	
8..11	Scale factor	Scale factor for this channel and mode as integer scaled by 1000*1000

GetAvailableMeasurementModesAck

This command is the response for *GetAvailableMeasurementModes* and lists all available measurement modes. The list contains all modes (RawData5, ...). Default measurement mode is on position 0 of the list.

0..3	Number of measurement modes	
4..X	Measurement mode	4 byte per mode (RawData5, ...)
...	...	

SensorStats

This command provides status of battery, BER and RSSI. The values are obtained from Bluetooth module.

0..3	Battery	Voltage in mV From iWRAP-documentation: <i>Current battery voltage in millivolts.</i>
4..7	BER	Bit Error Rate scaled with 1000 (to avoid floating values) From iWRAP-documentation: <i>Average Bit Error Rate on the link. Possible values are from 0.0000 to 100.0000.</i>
8..11	RSSI	Receiver Signal Strength Indication From iWRAP-documentation: <i>Receiver Signal Strength Indication. Possible values are from +20 to -128.</i> <i>20 = Good link</i> <i>-128 = Poor link</i>

SensorStatsExt

This command provides several status values and is an extended command of SensorStats. The values are obtained from Bluetooth module.

0..3	CSOC	State of charge in %
4..7	TTECP	Time to empty at constant power in minutes
8..11	AI	Average current in mA

12..15	RSOC	Relative state of charge in %
16..19	VOLT	Current voltage in mV

GetScaleFactorExt

This command retrieves scale factor for RawDataXX data and is an extended command of GetScaleFactor. Channel and mode are the same values as in *GetScaleFactorExt*. There are multiple modes.

0..3	Channel			
4..7	Mode			
8..9	Scale factor type			
10..25	Data depending on scale factor type	Scale factor mode	Byte	Description
		0	10..13	Used multiplier for scale value
			14..17	Scale value $ScaleFactor = \frac{ScaleValue}{ScaleMultiplier}$
		1	10..17	Scale factor as double value

ConfigStandBy

This command configures the sensor standby time. The set time is not stored permanently in flash, after reset/turn off the values is set back to default (5 minutes). If there is no measurement active and no communication occurred, the sensor turns off after configured time.

0..3	Time	In ms.
------	------	--------

MeasurementActiveAck

This command returns 1 if a measurement is running and 0 if no measurement is active.

0..3	Active	0 = not measurement active, 1 = measurement active
------	--------	--

GetCurrentSessionIdAck

This command returns the current session id. If no session is active, an error package is sent instead (ErrorTimerNotRunning).

0..3	Session id	Same as in PrepareMeasurement
------	------------	-------------------------------

GetUsbConnectStateAck

This command returns 0 if the sensor is not connected via USB and 1 if the sensor is powered from USB. A value of 2 indicate, that the device was enumerated.

0..3	USB connect state	0 usb is not connected, 1: usb is connected, 2: usb is configured
------	-------------------	---

GetLinkAck

This command returns 0 if the connection is via Bluetooth and 1 if the connection is via USB-CDC.

0..3	Link	0 Bluetooth, 1: USB-CDC
------	------	-------------------------

ChangeSendStorageMode

This command changes SendStorageMode even during an active measurement. The values for SendStorageMode can be found under Measurement mode of sensors.

0..3	SendStorageMode	Send and storage mode
------	-----------------	-----------------------

RawData3

The accelerometer provides 3*14 bits, gyroscope 3*16 bits and magnetometer 3*12 bit as 2-complement. The sensor data is minimal small packed to reduce space. 2 bits after MagZ are empty. The following table shows an overview of 16 bytes per sample. 16 samples are grouped in 1 package, this mean the whole RawData3 package is 4+16*16=260 bytes long.

These values must be converted with calibration formulas to physical units (see chapter Conversion of measurement data (packages: RawDataXX)).

0..3	Package number	Unique id for each measurement package
4..19	Sample 0	Measurement data
20..35	Sample 1	Measurement data
...	...	
244..259	Sample 15	Measurement data

Byteof sample	Content	
0	GyroX 8..15	
1	GyroX 0..7	
2	GyroY 8..15	
3	GyroY 0..7	
4	GyroZ 8..15	
5	GyroZ 0..7	
6	AccX 6..13	
7	AccX 0..5	AccY 12..13
8	AccY 4..11	
9	AccY 0..3	AccZ 10..13
10	AccZ 2..9	
11	AccZ 0..1	MagX 6..11
12	MagX 0..5	MagY 10..11
13	MagY 2..9	
14	MagY 0..1	MagZ 6..11

15	MagZ 0..5	
----	-----------	--

RawData4

This measurement package is equal to RawData3, besides only 6 samples instead of 16 samples are grouped together. This mean the whole RawData4 package is $4+6*16=100$ bytes long.

RawData5

The accelerometer provides $3*16$ bits, gyroscope $3*16$ bits and magnetometer $3*12$ bit as 2-complement. The sensor data is minimal small packed to reduce space. 4 bits after Mag are empty. The following table shows an overview of 18 bytes per sample. 14 samples are grouped in 1 package, this mean the whole RawData5 package is $4+14*18=256$ bytes long.

These values must be converted with calibration formulas to physical units (see chapter Conversion of measurement data (packages: RawDataXX)).

0..3	Package number	Unique id for each measurement package
4..21	Sample 0	Measurement data
22..39	Sample 1	Measurement data
...	...	
238..255	Sample 14	Measurement data

Byte of sample	Content
0	AccX 0..7
1	AccX 8..15
2	AccY 0..7
3	AccY 8..15
4	AccZ 0..7
5	AccZ 8..15
6	GyroX 0..7
7	GyroX 8..15
8	GyroY 0..7
9	GyroY 8..15
10	GyroZ 0..7
11	GyroZ 8..15
12	MagX 0..7
13	MagX 8..15
14	MagY 0..7
15	MagY 8..15
16	MagZ 0..7
17	MagZ 8..15

RawData6

This measurement package is equal to RawData5, besides only 6 samples instead of 14 samples are grouped together. This mean the whole RawData6 package is $4+6*18=112$ bytes long.

RawData7

This measurement package is equal to RawData5, besides only 1 sample instead of 14 samples is grouped together. This mean the whole RawData7 package is $4+1*18=22$ bytes long.

RawData8

This measurement package is equal to RawData7, besides it contains additionally a timestamp of the sample.

These values must be converted with calibration formulas to physical units (see chapter Conversion of measurement data (packages: RawDataXX)).

0..3	Package number	Unique id for each measurement package
4..21	Sample	Measurement data
22..25	Time	Timestamp of sample in ms

SyncData5

This measurement packages is equal to RawData5. Instead of the internal clock an external clock is used to measure data, for details see chapter Operation modes.

RawData9

The accelerometer provides $3*16$ bits, gyroscope $3*16$ bits and magnetometer $3*16$ bit as 2-complement. The sensor data is minimal small packed to reduce space. The following table shows an overview of 18 bytes per sample. 14 samples are grouped in 1 package, this mean the whole RawData9 package is $4+14*18=256$ bytes long.

These values must be converted with calibration formulas to physical units (see chapter Conversion of measurement data (packages: RawDataXX)).

0..3	Package number	Unique id for each measurement package
4..21	Sample 0	Measurement data
22..39	Sample 1	Measurement data
...	...	
238..255	Sample 14	Measurement data

Byte of sample	Content
0	AccX 0..7
1	AccX 8..15
2	AccY 0..7

3	AccY 8..15
4	AccZ 0..7
5	AccZ 8..15
6	GyroX 0..7
7	GyroX 8..15
8	GyroY 0..7
9	GyroY 8..15
10	GyroZ 0..7
11	GyroZ 8..15
12	MagX 0..7
13	MagX 8..15
14	MagY 0..7
15	MagY 8..15
16	MagZ 0..7
17	MagZ 8..15

RawData10

This measurement package is equal to RawData9, besides only 1 sample instead of 14 samples is grouped together. This mean the whole RawData10 package is $4+1*18=22$ bytes long.

RawData11

The accelerometer provides $3*16$ bits, gyroscope $3*16$ bits, magnetometer $3*16$ bit and pressure $1*24$ bit as 2-complement. The sensor data is minimal small packed to reduce space. The following table shows an overview of 21 bytes per sample. Each sample is send as package, this mean the whole RawData11 package is $4+21=25$ bytes long.

These values must be converted with calibration formulas to physical units (see chapter Conversion of measurement data (packages: RawDataXX)).

Byte of sample	Content
0	AccX 0..7
1	AccX 8..15
2	AccY 0..7
3	AccY 8..15
4	AccZ 0..7
5	AccZ 8..15
6	GyroX 0..7
7	GyroX 8..15
8	GyroY 0..7
9	GyroY 8..15
10	GyroZ 0..7
11	GyroZ 8..15

12	MagX 0..7
13	MagX 8..15
14	MagY 0..7
15	MagY 8..15
16	MagZ 0..7
17	MagZ 8..15
18	Pressure 0..7
19	Pressure 8..15
20	Pressure 16..23

RawData12

The accelerometer provides 3*16 bits, gyroscope 3*16 bits, magnetometer 3*16 bit and temperature 1*16 bit. The sensor data is minimal small packed to reduce space. The following table shows an overview of 20 bytes per sample. Twelve samples are send as a package including a dummy of twelve bytes at the end, this mean the whole RawData12 package is $4+12*20+12=256$ bytes long.

These values must be converted with calibration formulas to physical units (see chapter Conversion of measurement data (packages: RawDataXX)).

Byte of sample	Content
0	AccX 0..7
1	AccX 8..15
2	AccY 0..7
3	AccY 8..15
4	AccZ 0..7
5	AccZ 8..15
6	GyroX 0..7
7	GyroX 8..15
8	GyroY 0..7
9	GyroY 8..15
10	GyroZ 0..7
11	GyroZ 8..15
12	MagX 0..7
13	MagX 8..15
14	MagY 0..7
15	MagY 8..15
16	MagZ 0..7
17	MagZ 8..15
18	Temperature 0..7
19	Temperature 8..15

RawData13

This measurement package is equal to RawData12, besides only one sample instead of 12 samples is grouped together. This mean the whole RawData13 package is $4+1*20=24$ bytes long.

RawData14

The accelerometer provides 3*16 bits and gyroscope 3*16 bits. The sensor data is minimal small packed to reduce space. The following table shows an overview of 12 bytes per sample. Twenty-one samples are send as a package, this mean the whole RawData14 package is $4+21*12=256$ bytes long.

These values must be converted with calibration formulas to physical units (see chapter Conversion of measurement data (packages: RawDataXX)).

Byte of sample	Content
0	AccX 0..7
1	AccX 8..15
2	AccY 0..7
3	AccY 8..15
4	AccZ 0..7
5	AccZ 8..15
6	GyroX 0..7
7	GyroX 8..15
8	GyroY 0..7
9	GyroY 8..15
10	GyroZ 0..7
11	GyroZ 8..15

RawData15

This measurement package is equal to RawData14, besides only one sample instead of 21 samples is grouped together. This mean the whole RawData15 package is $4+1*12=16$ bytes long.

RawData16

The accelerometer provides 3*16 bits, gyroscope 3*16 bits and temperature 1*16 bit. The sensor data is minimal small packed to reduce space. The following table shows an overview of 14 bytes per sample. Eighteen samples are send as a package, this mean the whole RawData16 package is $4+18*14=256$ bytes long.

These values must be converted with calibration formulas to physical units (see chapter Conversion of measurement data (packages: RawDataXX)).

Byte of sample	Content
0	AccX 0..7
1	AccX 8..15
2	AccY 0..7
3	AccY 8..15
4	AccZ 0..7
5	AccZ 8..15
6	GyroX 0..7

7	GyroX 8..15
8	GyroY 0..7
9	GyroY 8..15
10	GyroZ 0..7
11	GyroZ 8..15
12	Temperature 0..7
13	Temperature 8..15

RawData17

This measurement package is equal to RawData16, besides only one sample instead of 18 samples is grouped together. This mean the whole RawData13 package is $4+1*14=18$ bytes long.

CalibData

Attention: only for test cases

This package contains scaled and calibrated measurement values scaled by factor 1.000.000 (1000*1000) to avoid floating values.

Bytes	Content
0..3	AccX in m/s^2
4..7	AccY in m/s^2
8..11	AccZ in m/s^2
12..15	GyroX in $^\circ/\text{s}$
16..19	GyroY in $^\circ/\text{s}$
20..23	GyroZ in $^\circ/\text{s}$
24..27	MagX in Ga
28..31	MagY in Ga
32..35	MagZ in Ga

OrientationData

This package contains the orientation of sensor as normalized quaternion.

Bytes	Name	Data type	Dimension	Range	Description
0..3	Packet number	uint32	1		
4..7	w	float32	1	0..1	
8..11	x	float32	1	0..1	
12..15	y	float32	1	0..1	
16..19	z	float32	1	0..1	

AccComparisonData

Attention: only for test cases

This package contains accelerometer values of two separate accelerometer chips and allows a comparison between these two. Each axis has 16 bit and 16 samples are grouped to one package.

Bytes	Content
0-1	Acc X Sensor 1
2-3	Acc Y Sensor 1
4-5	Acc Z Sensor 1
6-7	Acc X Sensor 2
8-9	Acc Y Sensor 2
10-11	Acc Z Sensor 2

GaitphaseData

This package contains the current gait phase.

Bytes	Name	Data type	Dimension	Range	Description
0..3	Packet number	uint32	1		
4..4	Gait phase	uint8	1	0..3	0: foot flat phase 1: pre swing 2: swing 3: loading response

MeasPackageRequest

Use this command to request a measurement data package again. The sensor can store measurement data in build in SD-Card (if available). The response to this package can be positive (MeasPackageResponse) or negative (MeasPackageUnavailable).

0..3	Session id	Session id from PrepareMeasurementAck
4..7	Package number	Requested package number

MeasPackageRequestList

Use this command to request a list of measurement data packages again. The sensor can store measurement data in build in SD-Card (if available). The response to this package is positive (MeasPackageResponse) or negative (MeasPackageUnavailable) separately for each requested package number. Up to 25 packages can be requested at one time.

0..3	Session id	Session id from PrepareMeasurementAck
4..7	Package number count	Number of requested package numbers, maximum is 25
8..11	Package number	Requested package number
...	...	

MeasPackageRequestRange

Use this command to request a range of measurement data packages again. The sensor can store measurement data in build in SD-Card (if available). The response to this package is positive

(MeasPackageResponse) or negative (MeasPackageUnavailable) separately for each requested package number.

0..3	Session id	Session id from PrepareMeasurementAck
4..7	First package number	First package
8..11	Last package number	Last package

MeasPackageResponse

This command is the positive response for MeasPackageRequest.

0..3	Session id	Session id from PrepareMeasurementAck
4..7	Package number	Requested package number
8..11	Measurement mode	Measurement mode of requested package number, (RawData5, ...)
12..xx	Data	Measurement data (depending on measurement mode)

MeasPackageUnavailable

This command is the negative response for MeasPackageRequest. It means the requested package is no longer available.

0..3	Session id	Session id from PrepareMeasurementAck
4..7	Package number	Requested package number

SyncPrepare

Hint: Only implemented for test cases, use SyncBtClock instead!

This command contains the current time from master and does nothing besides sending a response. It allows measuring of time duration.

0..3	Absolute time	Time in ms
------	---------------	------------

SyncTime

Hint: Only implemented for test cases, use SyncBtClock instead!

This command contains the current time from master and sets this as sensor time.

0..3	Absolute time	Time in ms
------	---------------	------------

SyncOffset

Hint: Only implemented for test cases, use SyncBtClock instead!

This command should contain the half average travel time duration measured with SyncPrepare and adds this as an offset to current sensor time.

0..3	Time offset	Time in ms
------	-------------	------------

SyncBtClock

This command has no data and reads the current Bluetooth clock from Bluetooth module.

Hint: Sometimes the bt clock from sensor is around 300ms off in comparison to master bt clock.

SyncBtClockAck

This command is the response of SyncBtClock.

0..3	Time before sync	Time in ms
4..7	Time after sync	Time in ms

SetRtc

This command sets a real time clock to the sensor. Time is encoded as Coordinated Unix Time (seconds since 01.01.1970 00:00). Real time is stored volatile during runtime and is lost after sensor shut down. Default time starts at 01.01.1970 00:00. If BT clock is set to zero, transmission delay will be ignored.

0..3	Real time	Unix time
4..7	BT clock	BT clock value at the same time as real time

GetRtcAck

This command gets previously set real time from the sensor.

0..3	Real time	Unix time
4..7	BT clock	BT clock value at the same time as real time

SetBtSettings

This command allows configuration of the Bluetooth module. The parameters are used when a new connection is created, they don't affect the current connection.

0..3	Role	Bluetooth role (0: sensor is master, 1: sensor is slave)
4..7	MTU	Maximum transfer unit in bytes

GetBtSettingsAck

This command is the response of GetBtSettings and contains configuration of the Bluetooth module.

0..3	Role	Bluetooth role (0: sensor is master, 1: sensor is slave)
4..7	MTU	Maximum transfer unit in bytes

TestConnection

This command provides the ability to test the throughput of the connection. The data length of the response can be specified as parameter.

0..3	Size	Size in bytes for response, maximum is 512 bytes
------	------	--

TestConnectionAck

This command is the response for TestConnection and has the requested data length.

0..X	Dummy data	Data length as specified in TestConnection
------	------------	--

ErrorListGetAck

This package contains a list of error codes.

Bytes	Name	Data type	Dimension	Range	Description
0..3	Error count	uint32	1		Error count, size of package is 4+ errorcount*4
4..7	Error code 1	uint32	1	See Error constants	
...	...	uint32	1	See Error constants	

SdCardGetSectorCountAck

0..3	Total sector count	Total available sd-card sector count, 1 sector = 512 bytes, if total sector count equals zero, then there is no SD-card inserted
4..7	Free sector count	Free sd-card sector count, if free sector count equals zero, then the file system is not mounted (device is connected to USB) or the sd-card is full

SdCardGetSessionCountAck

This command gives the number of measurement sessions stored on internal storage (SD-card). A measurement session (if enabled by SetMeasurementMode) is started by PrepareMeasurement and ends at StopMeasurement/UnprepareMeasurement.

0..3	Measurement session count	Count of stored measurement sessions
4..7	Id	Id of first measurement session
7..11	Id	Id of second measurement session
...	Id	...

SdCardGetSessionInfo

0..3	Measurement session id	Obtained with SdCardGetSessionCountAck
------	------------------------	--

SdCardGetSessionInfoAck

0..3	Measurement mode	Measurement mode, (RawData5, ...)
4..7	Package count	Stored measurement packages count
8..11	Start time	In ms, same time returned by StartMeasurement
12..15	Stop time	In ms, same time returned by StopMeasurement

SdCardClearAll

This command deletes all data from internal SD-card. Attention: this action takes a lot of time (several seconds), if there are a lot measurements stored on disk.

SetMonitorSettings

This command sets a real time clock to the sensor (like SetRtc). Additionally the sensor is put in sleep mode until the sensor is connected to a PC via USB (sensor must be enumerated to disable sleep mode).

0..3	Real time	Unix time
4..7	BT clock	BT clock value at the same time as real time

SetOrientationAlgorithmParameter

This command sets all parameters for orientation algorithm at once.

Byte	Name	Data type	Dimension	Range	Description
0..0	gyroUpdateFrequency	uint8	1	10-100 Hz	
1..1	magAccUpdateFrequency	uint8	1	10-100 Hz	
2..17	initialQuaternion	float32	4	0..1	w, x, y, z
18..29	initialBias	float32	3	0..MAX_GYRO	
30..33	tauAcc	float32	1	0..X	
34..37	tauMag	float32	1	0..X	
38..41	zeta	float32	1	0..X	
42..45	rawRating	float32	1	0..X	
46..59	Reserved	-	-	-	For future use

GetOrientationAlgorithmParameterAck

This command gets all parameters for orientation algorithm at once. Structure is identical to SetOrientationAlgorithmParameter.

SetOrientationAlgorithmParameterSingle

This command sets a single parameter for orientation algorithm. Values for ParamType can be found in table below. Data type, dimension, range and description for each ParamType can be found under command SetOrientationAlgorithmParameter.

Byte	Name	Data type	Dimension	Range	Description
0..1	ParamType	uint16	1	0..7	
2..X	ParamValue	Depends on ParamType	Depends on ParamType		
X+1..33	Reserved	-	-	-	For future use

ParamType	Name
0	gyroUpdateFrequency
1	magAccUpdateFrequency
2	initialQuaternion
3	initialBias
4	tauAcc
5	tauMag
6	zeta
7	rawRating

GetOrientationAlgorithmParameterSingle

This command gets a single parameter for orientation algorithm. Values for ParamType can be found under SetOrientationAlgorithmParameterSingle.

Byte	Name	Data type	Dimension	Range	Description
0..1	ParamType	uint16	1	0..7	

GetOrientationAlgorithmParameterSingleAck

This command gets a single parameter for orientation algorithm. Values for ParamType can be found under SetOrientationAlgorithmParameterSingle. Data type, dimension, range and description for each ParamType can be found under command SetOrientationAlgorithmParameter.

Byte	Name	Data type	Dimension	Range	Description
0..1	ParamType	uint16	1	0..7	
2..X	ParamValue	Depends on ParamType	Depends on ParamType		
X+1..33	Reserved	-	-	-	For future use

SetGaitPhaseAlgorithmParameter

This command sets all parameters for gait phase algorithm at once.

Byte	Name	Data type	Dimension	Range	Description
0..1	n_max_rbuf_noise	uint16	1	1-400	

2..5	noise_gain	float32	1	0..X	
6..7	nff	uint16	1	0..20	
8..9	pitch axis	int16	1	-3..3	
10..13	gamma0	float32	1	0..1	
14..17	veltoe0	float32	1	0..10	
18..21	j_hs0	float32	1	0..X	
22..25	velpercent	float32	1	0..X	
26..29	time in swing phase	float32	1	0..X	
30..31	m_avg	uint16	1	0..20	
32..35	st	float32	1	0..X	
36..47	sensor-to-toe vector	float32	3	0..X	
48..49	variable_nff	uint16	1	0..1	
50..63	Reserved	-	-	-	For future use

GetGaitPhaseAlgorithmParameterAck

This command gets all parameters for gait phase algorithm at once. Structure is identical to SetGaitPhaseAlgorithmParameter.

SetGaitPhaseAlgorithmParameterSingle

This command sets a single parameter for gait phase algorithm. Values for ParamType can be found in table below. Data type, dimension, range and description for each ParamType can be found under command SetGaitPhaseAlgorithmParameter.

Byte	Name	Data type	Dimension	Range	Description
0..1	ParamType	uint16	1	0..7	
2..X	ParamValue	Depends on ParamType	Depends on ParamType		
X+1..33	Reserved	-	-	-	For future use

ParamType	Name
0	n_max_rbuf_noise
1	noise_gain
2	nff
3	pitch axis
4	gamma0
5	veltoe0
6	j_hs0
7	velpercent
8	time in swing phase
9	m_avg
10	st
11	sensor-to-toe vector
12	variable_nff

GetGaitPhaseAlgorithmParameterSingle

This command gets a single parameter for gait phase algorithm. Values for ParamType can be found under SetGaitPhaseAlgorithmParameterSingle.

Byte	Name	Data type	Dimension	Range	Description
0..1	ParamType	uint16	1	0..7	

GetGaitPhaseAlgorithmParameterSingleAck

This command gets a single parameter for orientation algorithm. Values for ParamType can be found under SetGaitPhaseAlgorithmParameterSingle. Data type, dimension, range and description for each ParamType can be found under command SetGaitPhaseAlgorithmParameter.

Byte	Name	Data type	Dimension	Range	Description
0..1	ParamType	uint16	1	0..7	
2..X	ParamValue	Depends on ParamType	Depends on ParamType		
X+1..33	Reserved	-	-	-	For future use

Measurement mode of sensors

This chapter describes the modes and channels for each of the commands. These are used for various commands. Channel describes the sensor chip (accelerometer, gyroscope, magnetic field sensor) and mode defines the measurement range. Default mode is marked with a *. The SendStorageMode allows configuration when measurement data is send over Bluetooth or stored on the internal SD-card.

Channel	Description
0	Acc
1	Gyro
2	Mag
3	Pressure

Mode Acc	Description
0	1g
1	2g
2	4g
3*	8g
4	16g

Mode Gyro	Description
0	250°/s
1	500°/s
2*	1000°/s
3	2000°/s

Mode Mag	Description
0	0,88Gs
1*	1,3Gs
2	1,9Gs
3	2,5Gs
4	4,0Gs
5	4,7Gs
6	5,6Gs
7	8,1Gs

Mode Pressure	Description
0*	260-1260hPa

SendStorageMode	Description
0*	Send and not store
1	Send and store
2	Not send and store

Sensor positions

Each sensor has a predefined fixed position in order to know on which body part the sensor was attached. The position of the sensor can be read on the keypad left to the LED.

Value	Name	Description	Marker
0	spFootLeft	Foot left	Green L
1	spFootRight	Foot right	Green R
2	spShankLeft	Shank left	Orange L
3	spShankRight	Shank right	Orange R
4	spThighLeft	Thigh left	Blue L
5	spThighRight	Thigh right	Blue R
6	spPelvis	Pelvis	Lila M
7	spSternum	Sternum	
8	spWristLeft	Wrist left	
9	spWristRight	Wrist right	
10	spPelvisDay	Pelvis	
11	spPelvisNight	Pelvis	

Error constants

Code hex	Code decimal	Description	Reason
0x01	1	ErrorBadPackage	Package structure broken
0x02	2	ErrorWrongChecksum	Package checksum wrong
0x03	3	ErrorBadData	Wrong data size or
0x04	4	ErrorUnknownPackage	Command number unknown
0xA0	160	ErrorIwrapNoAnswer	
0xB0	176	ErrorInputBufferFull	
0xB1	177	ErrorInputBufferUnderrun	
0xB2	178	ErrorInputBufferOverrun	
0xB3	179	ErrorInputBufferSplit	
0xC0	192	ErrorMuxRead	
0xD0	208	ErrorCalibrationWrite	
0xD1	209	ErrorCalibrationRead	
0xE0	224	ErrorStoreWrite	
0xE1	225	ErrorStoreRead	
0xE4	228	ErrorSdCardUnavailable	SD-card not inserted or mounted
0xE5	229	ErrorSdCardSessionNotValid	
0xE6	230	ErrorSdCardCreateSessionFailed	
0xF0	240	ErrorMeasModePrepared	Sensor already prepared when command PrepareMeasurement called
0xF1	241	ErrorMeasModeNotPrepared	Sensor not prepared when command UnPrepareMeasurement called
0xF2	242	ErrorTimerRunning	
0xF3	243	ErrorTimerNotRunning	
0xF4	244	ErrorMeasCacheOverrun	
0xF5	245	ErrorSync	Unable to sync sensor with master bt clock
0x0E00	3584	ErrorInitBma180	
0x0E01	3585	ErrorInitIimu3000	
0x0E02	3586	ErrorInitHmc5883	
0x0E03	3587	ErrorInitMpu6050	
0x0E04	3588	ErrorInitMpu9150	
0x0E05	3589	ErrorInitLis331hh	
0x0E06	3590	ErrorInitSdCard	

Example of use

In this chapter, an exemplary sequence of commands for communication with sensors is described. The normal written commands are used to start and stop a measurement and are mandatory. Commands marked in italics in the table are optional. Once a measurement is started with StartMeasurement, the sensor automatically sends data packets until a StopMeasurement. The commands described here are typically to implement. As data packet should be used RawData5.

To achieve the lowest possible time lag in the use of multiple sensors, it is advisable to send the command StartMeasurement as synchronized as possible to each sensor and not to wait for the response StartMeasurementAck. An offline synchronization of measurement data can be achieved by using start and stop time from StartMeasurement and StopMeasurement.

Master	Sensor	Hints
Init		Initializes connection, call once at the beginning of the connection
	InitAck	
		Place settings and get sensor information, required once per connection
<i>GetSerial</i>		
	<i>GetSerialAck</i>	
<i>GetPosition</i>		
	<i>GetPositionAck</i>	
<i>GetAvailableMeasurementModes</i>		
	<i>GetAvailableMeasurementModesAck</i>	
<i>GetScaleFactor {for Acc}</i>		
	<i>GetScaleFactorAck</i>	
<i>GetScaleFactor {for Gyro}</i>		
	<i>GetScaleFactorAck</i>	
<i>GetScaleFactor {for Mag}</i>		
	<i>GetScaleFactorAck</i>	
<i>GetCalibrationData {for Acc}</i>		
	<i>GetCalibrationDataAck</i>	
<i>GetCalibrationData {for Gyro}</i>		
	<i>GetCalibrationDataAck</i>	
<i>GetCalibrationData {for Mag}</i>		
	<i>GetCalibrationDataAck</i>	
<i>SetMeasurementMode</i>		
	<i>SetMeasurementModeAck</i>	
PrepareMeasurement		Preparing for measurement, must be called each time before StartMeasurement
	PrepareMeasurementAck	
StartMeasurement		Start measurement
	StartMeasurementAck	
	MeasPackage	Measurement packages (RawData5, ...)
	MeasPackage	
	...	

	MeasPackage	
StopMeasurement		Stop measurement, call of UnprepareMeasurement not required as StopMeasurement makes this implicitly
	StopMeasurementAck	

Measurement packages

The sensor offers several measurement data packages that are to be used depending on the requirement. The available measurement data packages can be queried via `GetAvailableMeasurementModes`. In order to achieve high sampling rates, the use of `RawDataXXX` measurement data packages are recommended, as they combine several measured samples in a single package, thus saving bandwidth.

Availability

Because of changes to the board design and the device's sensor chips, each sensor does not support any `RawDataXXX` mode because these are based exactly on the raw values provided by the sensor chips. The following table summarizes the available modes.

Board revision	Available modes	Recommended maximum sample rate [Hz]
0.9	RawData3, RawData4	600
	CalibData	100
1.14	RawData5, RawData6	600
	CalibData	100
1.20	RawData5, RawData6	600
	CalibData	100
1.21	RawData5, RawData6	600
	CalibData, RawData7, RawData8	100
3.x	RawData9-10, RawData12-17	500
	OrientationData	100

Bandwidth usage

Each type of the measured data packets has a different need for the required bandwidth. The following table lists required bandwidth. Under sample are all values (acc, gyro, mag, 3-axis each) for a measurement time point understood.

Measurement package	Bytes per sample (incl. overhead)	Bandwidth 100Hz [kiB/s]	Bandwidth 600Hz [kiB/s]
RawData3	≈ 17	1,6	10,0
RawData4	≈ 18	1,8	10,5
RawData5	≈ 19	1,9	11,1
RawData6	≈ 20	2,0	11,7
RawData7	≈ 28	2,7	16,4
RawData8	≈ 32	3,1	18,8
CalibData	≈ 44	4,3	25,8

Sample rate fluctuations

The sensor provides not 100% of the set sampling rate, usually this does not deviate more than 1%. For `RawDataXX` modes with a sample rate of 600Hz around 597Hz are achieved, for 100Hz it is about 99Hz.

Conversion of measurement data (packages: RawDataXX)

Conversion of LSB values to physical values

$$scale_LSB_{acc} = \frac{LSB}{scale_{acc}} [unit: m/s^2]$$

$$scaled_LSB_{gyro} = \frac{LSB}{scale_{gyro}} [unit: Grad/s]$$

$$scale_LSB_{mag} = \frac{LSB}{scale_{mag}} [unit: Ga]$$

The scaling factors can be obtained via *GetScaleFactor* command.

Apply calibration data

Calibration data consists of a bias, rotation matrix and Cg1 and Cg2.

The structure of the rotation matrix is:

$$RotMat = \begin{pmatrix} xx & xy & xz \\ yx & yy & yz \\ zx & zy & zz \end{pmatrix}$$

Applying calibration data is done by using the following two formulas:

$$tmp = (scaled_LSB - bias) * RotMat$$
$$value_{calib} = tmp * Cg1 + tmp^2 * Cg2$$

Neutral calibration

The neutral calibration (does not change measurement values) is:

$$RotMat_{xx,yy,zz} = 1.0$$

$$Cg1_{x,y,z} = 1.0$$

$$All\ other\ values = 0.0$$

Bandwidth estimation RawData5

Stuffing probability

The probability to stuff a byte is assumed with 3%. Based on observation that the data stream has more small than large number and the stop byte is also a small number (0x0F), the stuffing probability is assumed slightly higher. That means an overhead of about 3 additional bytes per 100 bytes data.

$$\frac{(\text{Number special bytes})}{(\text{Number possible byte values})} = \frac{3}{256} \approx 1,1\% \rightarrow \text{flat } 3\%$$

Overhead from package structure

start byte + 2 bytes checksum + 2 bytes command + stop byte + 4 bytes package number
= 10 additional bytes

$$\frac{10 \text{ bytes overhead}}{(252 \text{ bytes data} + 10 \text{ bytes overhead})} \approx 3\% \text{ total overhead}$$

Used bandwidth

This calculation is for transmission measurement data from 1 sensor with 600Hz sample rate.

Measurement data:

$$(18 \text{ bytes per sample}) * (14 \text{ samples per package}) = 252 \text{ bytes}$$

Package size:

$$(6 \text{ bytes Header}) + (4 \text{ bytes package number}) + (252 \text{ bytes data}) \\ = (262 \text{ bytes per package})$$

$$(262 \text{ bytes per package}) + (3\% \text{ stuffing probability}) \approx 270 \text{ bytes data stream}$$

Bandwidth:

$$270 \text{ bytes} * (600\text{Hz sample rate}) \approx 11,3\text{kiB/s}$$

Sensor coordinate system

The used sensor coordinate system is shown below:



Time synchronization

The sensor counts its time with a millisecond resolution and begins with zero when turned on. There are 2 ways to synchronize sensor time with a master time.

Bluetooth clock

The Bluetooth clock is a 28-bit counter with 0.3125ms resolution. After around 23 hours it reaches its maximum and starts again from zero. The master Bluetooth device synchronizes this clock with all slaves in a piconet. To synchronize the sensor, call command SyncBtClock.

Under Linux Bluetooth clock can be obtained with command line tool *hcitool* with parameters *clock* and *client mac address*.

Time duration

The commands SyncPrepare, SyncTime and SyncOffset allow to implement time duration based method for synchronization similar to the network time protocol (NTP).

Hint: Only implemented for test cases, use SyncBtClock instead!

Operation modes

The sensor provides several predefined operating modes. The current mode can be set with command *SetSensorConfig* and retrieved with command *GetSensorConfig*. At the moment three modes are supported. The modes are stored in a bit mask as described below:

Bit	Description
0	External synchronization
1	Low latency optimized
2	Monitoring mode enabled

External synchronization

The sensor can use an external clock to measure data instead of the normal internal timer. The connection between an external clock and a sensor is done by cable. These steps are necessary:

- Connect (special configuration, please contact HASOMED) sensor version cables to GND and 3.3V
- Set measurement mode SyncData5 with command *SetMeasurementMode*
- Start measurement with commands *PrepareMeasurement* and *StartMeasurement*
- Generating input signal, measurement values is taken at rising edge of external signal
- Stop measurement with command *StopMeasurement*

Low latency optimization

In this operation mode, the internal processing timer runs with 500Hz and is independent from the selected measurement frequency. Additionally the Bluetooth sniff settings from the Bluetooth module are set to faster sniffing.

Monitoring mode

In this operation mode the sensor becomes an autonomous monitoring device. It starts and stops measuring depending on USB connection. It uses a predefined measurement mode and stores all data on internal sd-card. Retrieving measurement data is done via USB MSD functionality.

Measurement file format

This chapter describes the file format of a measurement stored on internal SD-card. If the sensor is plugged via USB to a PC, it reports a mass storage device with a FAT file system. To recognize a SD-card as valid, the sensor firmware creates file named "RW_FS" containing only the sensor serial number and position as four byte unsigned integer. If this file is missing during sensor initialization, a new file system will be created and any data is lost. Each measurement is a separate file. The name of the file consists of the session id without extension. Post processing is identical described in chapter Conversion of measurement data (packages: RawDataXX).

By default the sensor has no real time, this means that file date starts from 01.01.1970 00:00. Use command SetRtc before measurement start to adjust real time clock.

File structure index file

Bytes	Description
0..3	Serial
4..7	Position

File structure measurement file

Each measurement file starts with a header (512 bytes), followed by calibration data for accelerometer, gyroscope and magnetometer (512 bytes each) and continuing with raw measurement data.

Bytes	Description
0..511	Header
512..1023	Calibration data for accelerometer
1024..1535	Calibration data for gyroscope
1536..2047	Calibration data for magnetometer
2048..X	Measurement values, structure depends on selected measurement mode (RawData5, ...)

Header

Bytes	Description
0..3	Version of file format (current version: 1)
4..7	Session id (same as used as filename)
8..11	Measurement type (RawData5, ...)
12..15	Sample rate in Hz
16..19	Package count (not samples)
20..23	BT clock start time
24..27	BT clock stop time
28..31	Real time (must be set before measurement start with command SetRtc)
32..35	Sensor serial
36..39	Sensor position
40..43	Firmware version
44..47	Sensor configuration
48..511	Reserved

Calibration data

Structure of calibration data in measurement file is nearly equal to command SetCalibrationData/GetCalibrationData except 4 bytes in front for scale factor. All values are scaled by factor 1000*1000.

Bytes	Description
0..3	Scale factor
4..7	Channel
8..11	Mode
12..15	Bias X
16..19	Bias Y
20..23	Bias Z
24..27	Matrix XX
28..31	Matrix XY
32..35	Matrix XZ
36..39	Matrix YX
40..43	Matrix YY
44..47	Matrix YZ
48..51	Matrix ZX
52..55	Matrix ZY
56..59	Matrix ZZ
60..63	Cg1 X
64..67	Cg1 Y
68..71	Cg1 Z
72..75	Cg2 X
76..79	Cg2 Y
80..83	Cg2 Z
84..511	Reserved

Measurement data

Structure of measurement data depends on selected measurement mode including package number.